

Politechnika Gdańska
Wydział Elektroniki, Telekomunikacji i Informatyki
Katedra Teleinformatyki

Mediacja Transferu Płatnych Usług
Świadczonych przez Zawodne
Urządzenia Autonomiczne

Raport z Projektu — Testowanie Agentów AI

Modelowanie Procesów Ekonomicznych

Autorzy: Mikołaj Rutkowski (nr indeksu: 175534)
Stanisław Nagórski (nr indeksu: 202231)

Prowadzący: dr inż. Jerzy Konorski

Data: 30 kwietnia 2026

Spis treści

1	Cel i Zakres Projektu	2
2	Model Matematyczny Systemu	2
2.1	Opis systemu	2
2.2	Model stanu urządzenia	2
2.3	Model niezawodności i kosztów	3
2.4	Funkcja waluacji Konsumentów	3
2.5	Polityka transferu bufora (SPH-STP)	3
2.6	Dystrybucja płatności (SPH-PDP)	3
2.7	Oczekiwany zysk netto urządzenia	3
3	Metodologia Testowania Agentów AI	4
3.1	Narzędzie testowe: SPH Simulator	4
3.2	Prompt dla agenta AI	4
3.3	Przebieg iteracyjny	5
3.4	Framework GSD	5
4	Strategia Naiwna jako Punkt Odniesienia (Baseline)	5
4.1	Definicja strategii naiwnej	5
4.2	Optymalizacja parametru ζ	5
4.3	Wyniki strategii naiwnej	6
5	Wyniki Testowania Agentów AI	6
5.1	Agent 1: GitHub Copilot (Claude Sonnet 4.6)	6
5.1.1	Przebieg procesu decyzyjnego	6
5.1.2	Zaproponowane strategie	7
5.1.3	Weryfikacja zgodności motywacyjnej	7
5.2	Agent 2: Model referencyjny	7
6	Analiza Porównawcza	8
6.1	Zestawienie wszystkich wariantów	8
6.2	Pareto-front: waluacja a zysk netto	8
6.3	Analiza strategii odrzuconych	9
7	Podsumowanie Użycia Frameworka GSD	9
7.1	Zaangażowane modele i role	9
7.2	Przebieg etapów GSD	10
8	Wnioski	10
A	Komendy uruchomienia symulatora	10
B	Parametry modelu	11

1 Cel i Zakres Projektu

Celem projektu było opracowanie **środowiska testowego do oceny skuteczności agentów AI** w zadaniu projektowania reguły decyzyjnej dla modułu SPH systemu mediacji transferu płatnych usług. Zaimplementowany symulator (SPH Simulator) odwzorowuje mechanikę opisaną w specyfikacji projektu prof. J. Konorskiego.

Kluczowe założenia projektu:

- Opracowanie symulatora implementującego pełen model matematyczny systemu SPH/SUS.
- Sformułowanie szczegółowego *promptu* dla agentów AI opisującego problem optymalizacji.
- Przeprowadzenie wieloiteracyjnych sesji testowych z agentami AI (model Claude Sonnet 4.6 via GitHub Copilot oraz model referencyjny).
- Analiza wyników i porównanie strategii znalezionych przez agentów ze strategią naiwną (punkt odniesienia).

2 Model Matematyczny Systemu

2.1 Opis systemu

System składa się z:

- n_U zawodnych urządzeń zdolnych do dostarczania jednostek usług,
- modułu SPH (*Service and Payment Handling*) odpowiedzialnego za transfer usług i płatności,
- bufora SUS (*Service Units Storage*) o pojemności n_{SUS} jednostek usług,
- społeczności Konsumentów płacących za dostarczone usługi.

2.2 Model stanu urządzenia

Każde urządzenie $j \in \{1, \dots, n_U\}$ w cyklu t jest w jednym z dwóch trybów: UP (gotowość do pracy) lub DOWN (naprawa/utrzymanie). Urządzenie w trybie UP jest w *fazie* $i \in \{1, \dots, F - 1\}$ i podejmuje jedną z decyzji:

COMMIT — podjęcie zobowiązania do dostarczenia jednostki usług. Urządzenie ponosi koszt κ_i , a następnie:

- z prawdopodobieństwem φ_i doznaje awarii, przechodzi w tryb DOWN na jeden cykl (naprawa, koszt ϱ_i), po powrocie wchodzi w fazę 1,
- z prawdopodobieństwem $1 - \varphi_i$ pomyślnie dostarcza usługę (staje się *dostawcą*), otrzymuje płatność i przechodzi do fazy $i + 1$.

ABSTAIN — rezygnacja. Brak kosztów. Urządzenie przechodzi w tryb DOWN na jeden cykl (procedury utrzymania), po powrocie wchodzi w fazę 1.

2.3 Model niezawodności i kosztów

Parametry przyjęte zgodnie ze specyfikacją projektu:

$$0 < \varphi_1 \leq \varphi_2 \leq \dots \leq \varphi_F = 1, \quad F < \infty, \quad (1)$$

$$0 \leq \kappa_1 \leq \kappa_2 \leq \dots \leq \kappa_F, \quad 0 \leq \varrho_1 \leq \varrho_2 \leq \dots \leq \varrho_F. \quad (2)$$

Dla przyjętych danych wejściowych: $F = 5$, $\boldsymbol{\varphi} = (0.1, 0.2, 0.3, 0.4, 1.0)$, $\boldsymbol{\varrho} = (0.5, 0.5, 0.7, 1.5, 3.0)$, $\kappa = 0.25$ (stały dla wszystkich faz). Urządzenie w fazie $F = 5$ ma $\varphi_F = 1$, zatem *nigdy* nie podejmuje decyzji COMMIT.

2.4 Funkcja waluacji Konsumentów

Waluacja Konsumentów zależy od liczby u dostarczonych jednostek usług (po korekcie buforem SUS) w danym cyklu:

$$g(u) = \begin{cases} K_0 & \text{jeśli } K_0 \leq u \leq K_1, \\ 0 & \text{w pozostałych przypadkach,} \end{cases} \quad (3)$$

gdzie $K_0 = 100$, $K_1 = 120$. Waluacja wynosi 100 wyłącznie wtedy, gdy liczba dostawców po korekcie SUS mieści się w *oknie waluacji* $[K_0, K_1]$.

2.5 Polityka transferu bufora (SPH-STP)

Moduł SPH w każdym cyklu wyznacza wartości z^* (jednostki odkładane do SUS) i y^* (jednostki pobierane z SUS) tak, aby zmaksymalizować łączne wpływy P_s :

$$P_s = \max_x [g(u - x) + x], \quad x = z - y, \quad (4)$$

gdzie $x \in [-s, \min\{u, n_{\text{SUS}} - s + y\}]$, przy czym s oznacza zajętość SUS na początku cyklu. Wynikowa optymalna pula płatności dla dostawców wynosi $P^* = P_s$.

2.6 Dystrybucja płatności (SPH-PDP)

Płatność dla dostawcy w fazie i przy wektorze $\mathbf{l} = (l_i)_{i \in I}$ dostawców wynosi:

$$p_i(\mathbf{l}) = \frac{h(i)}{\sum_{j \in I} h(j) l_j} \cdot P^*, \quad (5)$$

gdzie $h(i) = i^\alpha$ ($\alpha = 1$ w przyjętych danych), zatem dostawca w fazie i dostaje i -krotność udziału dostawcy w fazie 1. Motywuje to urządzenia do commitowania w wyższych fazach mimo rosnącego ryzyka awarii.

2.7 Oczekiwany zysk netto urządzenia

Oczekiwany zysk netto urządzenia w fazie i przy decyzji COMMIT wynosi:

$$\mathbb{E}[\pi_i \mid \text{COMMIT}] = (1 - \varphi_i) \mathbb{E}[p_i] - \kappa - \varphi_i \varrho_i, \quad (6)$$

a przy decyzji ABSTAIN: $\mathbb{E}[\pi_i \mid \text{ABSTAIN}] = 0$.

Warunek *zgodności motywacyjnej* (IC, *Incentive Compatibility*) dla fazy i :

$$\boxed{\mathbb{E}[\pi_i \mid \text{COMMIT}] > 0 \iff (1 - \varphi_i) \mathbb{E}[p_i] > \kappa + \varphi_i \varrho_i.} \quad (7)$$

Reguła rekomendacji jest *zgodna motywacyjnie*, jeżeli warunek (7) jest spełniony dla każdej fazy, w której rekomendowane jest COMMIT.

3 Metodologia Testowania Agentów AI

3.1 Narzędzie testowe: SPH Simulator

Zaimplementowano symulator `sph_sim.py` w języku Python 3 (biblioteka standardowa: `argparse`, `json`, `random`, `math`, `dataclasses`). Simulator oferuje pięć wymienionych strategii rekomendacji:

- naive** COMMIT z jednakowym prawdopodobieństwem ζ dla urządzeń w trybie UP, niezależnie od fazy.
- threshold** COMMIT wyłącznie dla urządzeń w fazach $i \leq i_{\max}$.
- phase_prob** COMMIT z odrębnym prawdopodobieństwem ζ_i per faza.
- incentive** COMMIT gdy $\mathbb{E}[\pi_i \mid \text{COMMIT}] > 0$ (kryterium (7)).
- adaptive** COMMIT z prawdopodobieństwem zależnym od zajętości s bufora SUS.

Reprodukowalność wyników zapewniono przez ustalenie ziarna losowego (`seed = 42`).

3.2 Prompt dla agenta AI

Agentom AI dostarczono szczegółowy prompt (plik `PROMPT_DLA_AGENTA.txt`) zawierający:

- pełny opis systemu SPH/SUS wraz z mechaniką cyklu,
- parametry modelu ($n_U, n_{\text{SUS}}, K_0, K_1, \boldsymbol{\varphi}, \boldsymbol{\varrho}, \kappa, \alpha, T$),
- opis dostępnych strategii i ich parametrów,
- metryki oceny (KPI) wraz z kierunkiem optymalizacji,
- wyniki bazowe (baseline) jako punkt odniesienia,
- wymagany format odpowiedzi: *nazwa strategii, uzasadnienie, gotowa komenda*.

Agent miał zaproponować *jedną* strategię i zwrócić *jedną* gotową komendę uruchomienia symulatora.

3.3 Przebieg iteracyjny

Testowanie przebiegało cyklicznie dla każdego agenta:

1. **Prezentacja promptu** — agent otrzymuje pełny opis systemu i metryki oceny.
2. **Wybór strategii** — agent proponuje strategię i parametry, zwraca komendę.
3. **Uruchomienie symulatora** — komenda jest wykonywana ($T = 1000$ cykli, $\text{seed}=42$).
4. **Przekazanie wyników** — agent otrzymuje zmierzone KPI z symulacji.
5. **Iteracja** — agent może zrewidować wybór lub zoptymalizować parametry.

Schemat ten stanowi uproszczoną pętlę *obserwacja—decyzja—ocena* typową dla systemów uczenia przez wzmacnianie (*reinforcement learning*).

3.4 Framework GSD

Do zarządzania sesją agenta wykorzystano framework **GSD** (*Get Shit Done*), strukturyzujący pracę asystenta AI w cykl:

Mapowanie $\longrightarrow$ *Inicjalizacja* $\longrightarrow$ *Planowanie* $\longrightarrow$ *Wykonanie* $\longrightarrow$ *Weryfikacja*.

W fazie mapowania cztery agenty Claude Haiku 4.5 analizowały kod symulatora równolegle (aspekty: stos technologiczny, architektura, konwencje, dług techniczny).

4 Strategia Naiwna jako Punkt Odniesienia (Baseline)

4.1 Definicja strategii naiwnej

Strategia naiwna (*naive*) rekomenduje COMMIT każdemu urządzeniu w trybie UP z jednakowym prawdopodobieństwem ζ , niezależnie od fazy:

$$\Pr(\text{COMMIT}_j^{(t)} = 1 \mid \text{UP}) = \zeta, \quad \forall j, \forall i \in \{1, \dots, F - 1\}. \quad (8)$$

Jest to punkt odniesienia dla wszystkich bardziej zaawansowanych reguł rekomendacji. Strategia spełnia kryterium niedyskryminacyjności (jednakowe traktowanie faz).

4.2 Optymalizacja parametru ζ

Docelowa liczba dostawców powinna mieścić się w oknie $[K_0, K_1]$. Przy $n_U = 250$ urządzeniach, w stanie ustalonym ≈ 125 – 140 urządzeń jest w trybie UP (pozostałe DOWN po poprzednim COMMIT lub ABSTAIN). Aby $u \in [100, 120]$, potrzeba ≈ 75 – 85% urządzeń UP z decyzją COMMIT, z uwzględnieniem awarii.

Empiryczna optymalizacja (grid search po $\zeta \in [0.70, 0.82]$) wykazała, że maksimum metryki $\bar{v}_{100}$ dla strategii naiwnej osiągnęte jest przy $\zeta^* \approx 0.75$ – 0.77 .

4.3 Wyniki strategii naiwnej

Tabela 1: Wyniki strategii naiwnej (baseline) przy $\zeta = 0.75$, $T = 1000$, $\text{seed}=42$.

Metryka	Symbol	Wartość
Śr. waluacja Konsumentów (ost. 100 cykli)	$\bar{v}_{100}$	92.00
Łączna waluacja ($T = 1000$ cykli)	V_T	92 000
Śr. zysk netto na urządzenie	$\bar{\pi}$	+140.76
Wskaźnik ciągłości dostaw	δ	79.31%
Śr. liczba dostawców (ost. 100 cykli)	$\bar{u}_{100}$	105.03

Strategia naiwna osiąga $\bar{v}_{100} = 92$, co oznacza, że 92% spośród ostatnich 100 cykli symulacji kończyło się waluacją $g(u) = 100$ (dostawcy w oknie $[100, 120]$). Jest to **punkt odniesienia** dla wszystkich badanych strategii.

5 Wyniki Testowania Agentów AI

5.1 Agent 1: GitHub Copilot (Claude Sonnet 4.6)

5.1.1 Przebieg procesu decyzyjnego

Agent przeanalizował problem i zaproponował strategię `phase_prob` jako najbardziej elastyczną, umożliwiającą różnicowanie prawdopodobieństwa COMMIT per fazę. Przeprowadził iteracyjną optymalizację metodą grid search (≈ 100 uruchomień symulatora w 2 iteracjach), poszukując konfiguracji maksymalizującej $\bar{v}_{100}$ przy $\bar{\pi} > 0$.

Iteracja 1 — optymalizacja waluacji:

1. Runda 1: przegląd wszystkich 5 strategii, 23 uruchomień. Wynik: strategie `adaptive` i `incentive` całkowicie nieefektywne (patrz Tab. 7).
2. Runda 2: fine-tuning `naive` i `phase_prob`; osiągnięto sufit $\bar{v}_{100} = 98$ dla `naive` ($\zeta = 0.768$).
3. Runda 3: optymalizacja parametrów `phase_prob`; odkryto konfigurację A ($\bar{v}_{100} = 99$).
4. Runda 4: weryfikacja — potwierdzono wynik.

Iteracja 2 — optymalizacja zysku przy zachowaniu $\bar{v}_{100} \geq 99$:

5. Rundy 5–8: przesunięcie COMMIT ku fazom 1–2 (niskie φ_i), odkrycie Pareto-frontu; odkryto konfigurację B bijącą oba wskaźniki modelu referencyjnego.

5.1.2 Zaproponowane strategie

Strategia `phase_prob` definiuje wektor prawdopodobieństw $(\zeta_1, \zeta_2, \zeta_3, \zeta_4, \zeta_5)$:

$$\Pr(\text{COMMIT}_j^{(t)} = 1 \mid \text{UP, faza } i) = \zeta_i, \quad \zeta_5 = 0 \text{ (zawsze ABSTAIN)}. \quad (9)$$

Trzy Pareto-optymalne konfiguracje znalezione przez agenta:

Tabela 2: Pareto-optymalne konfiguracje strategii `phase_prob` (Copilot).

Wariant	Probs $(\zeta_1, \dots, \zeta_4)$	$\bar{v}_{100}$	$\bar{\pi}$	δ	$\bar{u}_{100}$
A (maks. walucja)	0.84, 0.78, 0.70, 0.53	99	+153.99	80.76%	109.03
B (kompromis) [★]	0.94, 0.87, 0.34, 0.08	98	+194.09	83.70%	110.02
C (maks. zysk)	0.93, 0.87, 0.34, 0.08	97	+202.01	83.70%	109.04

[★] Rekomendowana: bije model referencyjny na obu głównych metrykach.

5.1.3 Weryfikacja zgodności motywacyjnej

Dla wariantu A ($\zeta = (0.84, 0.78, 0.70, 0.53, 0)$) empiryczna weryfikacja warunku (7) (na podstawie $T = 1000$ cykli):

Tabela 3: Zgodność motywacyjna (IC) wariantu A — dane empiryczne z symulacji.

Faza i	COMMIT	Sukces	$\mathbb{E}[p_i]$	$\mathbb{E}[c_i]$	$\mathbb{E}[\pi_i]$
1	58 839	90.0%	0.4216	0.2998	+0.1218
2	41 308	79.8%	0.7426	0.3509	+0.3917
3	23 094	70.1%	0.9676	0.4591	+0.5085
4	12 713	60.2%	1.1156	0.8471	+0.2685

Zgodność motywacyjna IC:

TAK (wszystkie fazy)

Warunek (7) jest spełniony dla każdej fazy: $\mathbb{E}[\pi_i \mid \text{COMMIT}] > 0$ dla $i \in \{1, 2, 3, 4\}$.

5.2 Agent 2: Model referencyjny

Drugi testowany agent (model referencyjny) zaproponował strategię o wyższym wskaźniku ciągłości dostaw ($\delta = 84.42\%$) kosztem niższej walucji:

Tabela 4: Wyniki modelu referencyjnego ($T = 1000$, `seed=42`).

Metryka	Symbol	Wartość
Śr. walucja Konsumentów (ost. 100 cykli)	$\bar{v}_{100}$	92.00
Łączna walucja ($T = 1000$ cykli)	V_T	91 200
Śr. zysk netto na urządzenie	$\bar{\pi}$	+189.50
Wskaźnik ciągłości dostaw	δ	84.42%
Śr. liczba dostawców (ost. 100 cykli)	$\bar{u}_{100}$	111.08
Końcowa zajętość SUS	s_T	20 (pełna)

Model referencyjny uzyskał $\bar{v}_{100} = 92$ — identycznie jak baseline naiwny — lecz przy znacznie wyższym zysku netto ($\bar{\pi} = +189.50$ vs. $+140.76$) i wyższym wskaźniku ciągłości dostaw.

6 Analiza Porównawcza

6.1 Zestawienie wszystkich wariantów

Tabela 5: Zestawienie wyników wszystkich badanych wariantów.

Wariant	$\bar{v}_{100}$	$\bar{\pi}$	δ	$\bar{u}_{100}$
<i>Punkt odniesienia</i>				
Naiwna (naive, $\zeta = 0.75$) [BASELINE]	92	+140.76	79.31%	105.03
<i>Model referencyjny</i>				
Model referencyjny	92	+189.50	84.42%	111.08
<i>GitHub Copilot (Claude Sonnet 4.6) — Pareto front</i>				
Wariant A (phase_prob 0.84,0.78,0.70,0.53)	99	+153.99	80.76%	109.03
Wariant B (phase_prob 0.94,0.87,0.34,0.08)	98	+194.09	83.70%	110.02
Wariant C (phase_prob 0.93,0.87,0.34,0.08)	97	+202.01	83.70%	109.04
<i>Strategie odrzucone</i>				
incentive (expected_P=90)	46	−107.64	79.00%	—
threshold (max_phase=3)	21	−170.43	81.00%	—
adaptive (s_target 5–15)	0	−34.48	88.00%	24.57

6.2 Pareto-front: waluacja a zysk netto

Badania wykazały fundamentalny **trade-off** między $\bar{v}_{100}$ a $\bar{\pi}$:

Tabela 6: Pareto-front (ziaren seed=42): kompromis waluacja – zysk.

ζ	$\bar{v}_{100}$	$\bar{\pi}$
(0.84, 0.78, 0.70, 0.53, 0)	99	+153.99
(0.94, 0.87, 0.34, 0.08, 0)	98	+194.09
(0.93, 0.87, 0.34, 0.08, 0)	97	+202.01
(0.90, 0.82, 0.37, 0.11, 0)	96	+203.25
(0.89, 0.82, 0.38, 0.12, 0)	95	+205.88

Źródło trade-offu: strategie koncentrujące COMMIT w fazach 1–2 (niskie φ_i) obniżają koszty awarii (wyższy $\bar{\pi}$), ale generują większą wariancję liczby dostawców per cykl, co skutkuje częstszym wychodzeniem poza okno $[100, 120]$ (niższe $\bar{v}_{100}$). Formalnie, przy strategii skupionej na fazach 1–2:

$$\text{Var}(u^{(t)}) \uparrow \Rightarrow \Pr(u^{(t)} \notin [K_0, K_1]) \uparrow \Rightarrow \mathbb{E}[g(u^{(t)})] \downarrow. \quad (10)$$

6.3 Analiza strategii odrzuconych

Tabela 7: Analiza strategii odrzuconych — przyczyny nieefektywności.

Strategia	Wynik	Przyczyna nieefektywności
adaptive	$\bar{v}_{100} = 0, \bar{u}_{100} \approx 25$	Bufor SUS (pojemność 20) zbyt mały, by stabilizować 250 urządzeń. Sygnał $s^{(t)}$ nie odzwierciedla potrzeby dostawców w oknie $[100, 120]$.
incentive	$\bar{v}_{100} = 46, \bar{\pi} < 0$	Zakładana pula płatności <code>expected_P</code> nie odpowiada dynamice systemu; urządzenia commitują zbyt agresywnie/pasywnie w zależności od fazy.
threshold	$\bar{v}_{100} = 21, \bar{\pi} < 0$	Twarde odcięcie faz $i > i_{\max}$ eliminuje zbyt wiele urządzeń, redukując $\bar{u}_{100}$ poniżej $K_0 = 100$.

7 Podsumowanie Użycia Frameworka GSD

7.1 Zaangażowane modele i role

Tabela 8: Modele AI zaangażowane w projekcie i ich role.

Model	Rola	Szczegóły
Claude Sonnet 4.6 (sesja główna)	Orchestrator, analityk, solver, grid search	1 sesja ciągła, ≈ 100 wywołań symulatora, interpretacja wyników, analiza Pareto-frontu
Claude Haiku 4.5 $\times 4$ (agenty mapujące)	Mapowanie kodu (izolowane konteksty)	Aspekty: tech/arch/quality/concerns; czas: ≈ 6 min równoległe; 7 dokumentów, 1082 linie

7.2 Przebieg etapów GSD

Tabela 9: Przebieg procesu GSD.

Etap	Działanie	Narzędzie GSD
Mapowanie kodu	4 agenty zmapowały kod równoległe (stack, arch, konwencje, dług techniczny)	/gsd-map-codebase
Inicjalizacja	Zebranie kontekstu projektu, wskazanie specyfikacji	/gsd-new-project
Planowanie + Wykonanie	Grid search ≈ 100 symulacji (naive/phase_prob/adaptive), fine-tuning	bash + sph_sim.py
Weryfikacja	Potwierdzenie wyników, analiza Pareto-frontu, dokumentacja	Końcowy run symulatora

8 Wnioski

1. **Skuteczność agentów AI.** Obydwa testowane agenty przekroczyły wynik strategii naiwnej ($\bar{v}_{100} = 92$) w zakresie co najmniej jednej metryki. Agent Copilot/Claude Sonnet 4.6 uzyskał $\bar{v}_{100} = 99$ (wariant A) i $\bar{\pi} = +194$ (wariant B).
2. **Strategia naiwna jako punkt odniesienia.** Strategia naiwna o $\zeta = 0.75$ osiąga $\bar{v}_{100} = 92$ — stanowi użyteczny, dobrze zrozumiany punkt odniesienia, lecz nie eksploatuje informacji o fazie urzędzenia.
3. **Przewaga strategii fazowo-zróżnicowanej.** Strategia phase_prob wyraźnie dominuje nad naive: różnicując ζ_i per faza, precyzyjniej utrzymuje dostawców w oknie $[K_0, K_1] = [100, 120]$.
4. **Fundamentalny trade-off.** Przy ustalonej konfiguracji symulatora (seed=42) nie istnieje strategia, która jednocześnie maksymalizuje $\bar{v}_{100}$ i $\bar{\pi}$. Pareto-front opisuje krzywa:

$$\bar{v}_{100} \nearrow 99 \iff \bar{\pi} \searrow 154, \quad \bar{\pi} \nearrow 202 \iff \bar{v}_{100} \searrow 97.$$

5. **Zgodność motywacyjna.** Wariant A spełnia warunek IC we wszystkich fazach ($\mathbb{E}[\pi_i \mid \text{COMMIT}] > 0$ dla $i \in \{1, 2, 3, 4\}$).
6. **Ograniczenia metodologii.** Wyniki są deterministyczne dla seed=42; nie przeprowadzono analizy wrażliwości na ziarno losowe. Brak testów jednostkowych symulatora może powodować niezauważone błędy numeryczne.

A Komendy uruchomienia symulatora

Poniższe komendy replikują wyniki zaprezentowane w raporcie:

Listing 1: Baseline: strategia naiwna

```
python sph_sim.py --strategy naive --zeta 0.75 \
  --nU 250 --nSUS 20 --K1 120 --T 1000 --kappa 0.25 --alpha 1
```

Listing 2: Warian A: maksymalna waluacja

```
python sph_sim.py --strategy phase_prob \
  --probs 0.84,0.78,0.70,0.53,0.0 \
  --nU 250 --nSUS 20 --K1 120 --T 1000 --kappa 0.25 --alpha 1
```

Listing 3: Warian B: kompromis (rekomendowany)

```
python sph_sim.py --strategy phase_prob \
  --probs 0.94,0.87,0.34,0.08,0.0 \
  --nU 250 --nSUS 20 --K1 120 --T 1000 --kappa 0.25 --alpha 1
```

Listing 4: Warian C: maksymalny zysk netto

```
python sph_sim.py --strategy phase_prob \
  --probs 0.93,0.87,0.34,0.08,0.0 \
  --nU 250 --nSUS 20 --K1 120 --T 1000 --kappa 0.25 --alpha 1
```

B Parametry modelu

Tabela 10: Pełna lista parametrów modelu przyjętych w projekcie.

Parametr	Symbol	Wartość	Opis
Liczba urządzeń	n_U	250	urządzenia
Pojemność SUS	n_{SUS}	20	jednostki usług
Maks. faza	F	5	—
Dolny próg waluacji	K_0	100	—
Górny próg waluacji	K_1	120	—
Koszt dostarczenia	κ	0.25	j.p. (stały)
Prob. awarii fazy 1–5	φ	(0.1, 0.2, 0.3, 0.4, 1.0)	—
Koszty naprawy fazy 1–5	ϱ	(0.5, 0.5, 0.7, 1.5, 3.0)	j.p.
Wykładnik dystrybucji	α	1	$h(i) = i^\alpha$
Liczba cykli	T	1000	—
Ziarno losowe	seed	42	—